

Progressive Retrieval Attention: Bounded Sparse Native-KV for Addressable Logical Memory

Jorge Simao

EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 12, 2026

Abstract

Transformer context windows usually make logical memory and active attention grow together. Progressive Retrieval Attention (PRA) separates them. A large logical memory is encoded once as layer-native key/value (K/V) state; compact gists route each query to a bounded subset; and only the selected, full-detail K/V participates in the model’s ordinary attention operation. The gist is therefore metadata for selection, not a compressed replacement for the token state that attention reads.

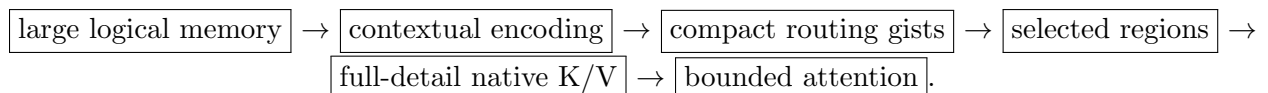
We evaluate this mechanism in controlled standalone transformers. The central repair is to encode contiguous context before slicing smaller routing units; independently encoded microchunks lose the historical context needed for faithful K/V. After this repair, fixed selection breadth over 64–256 addressable units reduces the active fraction from 17.9–18.7% to 4.4–4.9% while retaining essentially all measured dense-context benefit in the small model. A fixed-fraction reanalysis isolates router quality from increasing sparsity: selecting about 12.5% of references gives annotated target coverage of 0.756/0.772/0.762 on HotpotQA-derived examples and 0.794/0.803/0.800 on QASPER-derived examples at 64/128/256 units. Under a hard 32-token native-operation limit, PRA addresses a 184-token displaced prompt head inside 192 logical tokens, with loss 1.0567 versus 1.2287 for truncation and no operation-limit violations. Exact 256-way routing takes 5.5–11.6 ms after index construction in this prototype.

These results establish sparse transport of selected native K/V and bounded execution over larger logical memory. They do not establish unrestricted question answering, arbitrary positional extrapolation, or production serving efficiency.

1 Introduction

Long-context systems face two different questions that are often collapsed into one. First, how much information may the application logically address? Second, how many K/V tokens must a transformer operation process at once? Dense self-attention answers both with the same sequence. This is simple, but it makes active compute and device-resident state grow with the whole context even when a continuation depends on only a few earlier regions.

PRA separates those quantities. It stores historical model state outside the current token stream, uses compact routing gists to select relevant regions, and materializes the selected regions at full token resolution. The selected memory K/V and direct-context K/V then share the same attention softmax. In short:



This is not simply retrieval-augmented generation (RAG). RAG retrieves text and inserts it into a prompt; PRA determines how already encoded model state enters attention [5, 3, 2]. The two can be composed: a retriever may choose documents while PRA controls sparse access within their cached state. PRA is also distinct from sparse attention over one admitted sequence and from methods that discard or merge token K/V permanently. Its routing representation is compressed, but the selected payload is not.

The central claim of this paper is deliberately narrow:

PRA can make the memory logically visible to a decoder substantially larger than the K/V set active in any one model operation, while preserving the full-detail native K/V of the selected regions.

We test that claim with small, controlled PyTorch models rather than a production-scale language model. The experiments separate four issues that would otherwise be easy to confuse: whether cached native K/V carries useful detail, whether the router finds the annotated source, how quality changes as selection becomes sparse, and whether every underlying model call remains within a declared length bound.

Table 1: Headline results. Together, the rows separate representation fidelity, sparse systems scaling, routing quality, bounded execution, and device residency.

Finding	Measurement	Meaning
Fragmentation repair	Split-64 oracle RCB: .546→.996 (HotpotQA-derived), .748→.999 (QASPER-derived)	Encode contiguous context before exposing smaller routing views
Fixed- k sparse scaling	$k = 8$ keeps 11–13 physical K/V tokens active; active fraction falls .179→.044 and .187→.049 from 64 to 256 units	Accessible memory grows while the selected payload remains approximately flat
Fixed-fraction quality scaling	Coverage at about 12.5% selection is .756/.772/.762 and .794/.803/.800 at 64/128/256 units	Fixed- k decline partly reflects increasing sparsity rather than routing collapse
Bounded logical context	192 logical tokens, 184-token <code>#_head</code> , largest native call 20 under a limit of 32, zero violations	Logical memory can exceed one native operation without violating the configured bound
CPU-resident detail K/V	At 256 units, .734–3.420 MiB leaves GPU; loss and selection are unchanged; warm overhead is 4.0–13.5 ms	Sparse selection becomes a measured device-capacity tradeoff when inactive detail is offloaded
Measured capacity effect	QASPER-derived routed RCB at 256 units: .908 tiny versus 1.000 small	Some high-scale weakness belongs to the tested model/router capacity, not transport

The headline is not that one operating point solves long context. It is that logical memory, routing resolution, active native K/V, and device residency are separately controllable and separately measurable.

The remainder of the paper first explains the mechanism, then evaluates transport, routing, scaling, and bounded execution in that order. Positional extrapolation is treated only as a boundary here; RoPE-specific semantics belong to the companion position paper. Pretrained Hugging Face adaptation, iterative multi-hop retrieval, multi-gist routing, and production serving are separate extensions.

2 PRA in 60 Seconds

Assume an application has a long source and a short direct tail. PRA processes the source in model-safe contiguous encoding blocks. At each participating transformer layer it caches the resulting token K/V. Smaller routing chunks point to slices of those cached tensors, and each chunk receives a compact gist. At decode time, the current query scores the gists, a budgeter chooses chunks, and their original K/V slices join the direct K/V in one attention operation. Figure 1 shows the complete path.

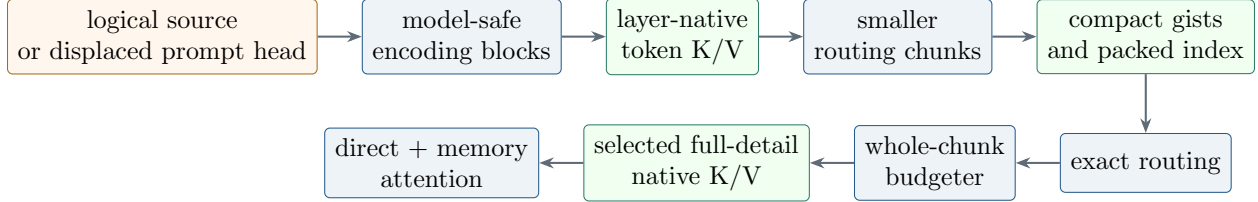


Figure 1: **PRA compresses the search representation, not the selected memory payload.** Contextual encoding produces native state, compact gists select addresses, and full-detail token K/V enters bounded attention.

Three lengths make the separation precise:

L_{logical}	The memory the request may logically address: direct history plus registered external or displaced source state.
$L_{\text{model-op}}$	The K/V tokens participating in one native transformer operation. PRA budgets this quantity independently of L_{logical} .
L_{position}	The coordinate assigned to a token when position-dependent transformations are applied. It is conceptually independent of both lengths above. This paper preserves known coordinates and does not claim arbitrary extrapolation.

For memory K/V (K_M, V_M) and direct K/V (K_D, V_D), the final operation is

$$\text{PRA}(Q) = \text{softmax}\left(\frac{Q[K_M; K_D]^\top}{\sqrt{d_h}} + M\right)[V_M; V_D], \quad (1)$$

where M preserves causal and padding constraints. The same attention scores and output projection are used for memory and direct state. The prototype’s default native-KV mode does not introduce a separate memory output projection. All main experiments evaluate this native-KV path. Appendix B records an earlier cross-attention-capable V1 branch only for historical provenance.

3 Mechanism

3.1 References, Gists, and Full-Detail Payloads

A reference identifies an application-authorized source. The source may be explicitly named, or it may be an implicit displaced prompt head. Encoding produces layer-native token K/V and records source coordinates. Routing chunks are metadata views over spans of that state. Each chunk contains a compact gist and offsets into the underlying K/V.

This distinction matters. If a mean gist represents 16 token positions, attention does not receive one averaged value in place of those positions. The gist participates only in ranking. If selected, the chunk’s 16 original key and value vectors are materialized. PRA therefore compresses the selection problem, not the content delivered to attention.

3.2 Encoding Blocks Are Not Routing Chunks

Addressable granularity does not need to equal encoding granularity. Consider a 128-token source region that should be searchable at 32-token resolution. PRA can encode all 128 tokens causally once, retain their contextual hidden and K/V states, and expose four 32-token views to the router. The model receives the context it needs, while the index retains fine addresses.

Fragmented path	128 source tokens → four independent 32-token model calls → four context-poor K/V blocks
Contextual-slice path	128 source tokens → one causal encoding → native hidden/K/V → four 32-token routing views

Figure 2: **Encoding resolution and routing resolution are independent.** Larger causal encoding preserves historical context; smaller metadata views preserve sparse selection granularity without re-encoding isolated microchunks.

An *encoding block* is therefore a contiguous model call that produces contextual state. A *routing chunk* is a smaller addressable view used by the index and budgeter. Encoding each routing chunk independently removes left context before K/V is cached, and no later router can reconstruct state the encoder never produced. For model-bounded sources, a block may carry overlapping left context while committing only its nonoverlapping core.

3.3 Exact Routing and Whole-Chunk Budgeting

For query summary q and gist g_i , the baseline router computes cosine-like scores,

$$s_i = \frac{q^\top g_i}{\max(\|q\|_2 \|g_i\|_2, \epsilon)}, \quad (2)$$

and selects top-ranked chunks exactly with tensorized **topk**. Exact means that all eligible candidates are scored; it does not mean oracle relevance. The implementation packs normalized gists once, reuses that index for later queries, and preserves a scalar reference implementation for parity tests.

Selection breadth alone does not determine active K/V because chunks may differ in size or overlap. The materializer admits whole chunks under a token budget. With a native operation limit $L_{\max}$,

$$L_D + L_M + L_R \leq L_{\max}, \quad (3)$$

where L_D is direct context, L_M materialized memory, and L_R a safety reserve. Whole-chunk admission keeps accounting auditable and avoids silently presenting partial evidence as a complete chunk.

3.4 What PRA Changes, and What It Does Not

PRA changes the set of historical K/V active for a query. It does not make external storage free, guarantee that the router selects causal evidence, or enlarge the positional range a pretrained model

has learned. It can reduce active attention and GPU residency while retaining a larger logical cache elsewhere. Those are systems benefits only when indexing, transfer, and cache construction are also measured.

Sparse-attention methods such as Longformer, BigBird, Reformer, and Routing Transformer change connectivity within an admitted sequence [1, 10, 4, 8]. KV-cache methods such as H₂O, SnapKV, and ClusterKV reduce retained decode state [11, 6, 7]. PRA instead retains source detail outside the active operation and uses a separate compact routing representation to decide which full token K/V to expose.

4 Evaluation Design

The standalone experiments use decoder-only transformers in two controlled capacity tiers: a tiny tier with 2.2–2.4M parameters and a small tier with 6.6–6.9M parameters. Unless otherwise stated, comparisons use five paired seeds and 32 held-out examples per seed. The goal is mechanism isolation, not benchmark leadership.

The datasets have different jobs:

- **Synthetic reference data** creates direct, known dependence on an earlier source span and exposes failures that natural language may hide.
- **WikiText-2** checks ordinary language-model behavior and contextual integrity. It is not used as evidence that a free-running model performs long-document QA.
- **HotpotQA-derived and QASPER-derived probes** provide natural text, distractors, and annotated source identity. Targets are controlled answer codes, so these experiments measure evidence transport and routing coverage rather than unrestricted QA.

We report loss and exact target accuracy where defined. For memory contribution, the recovered-context-benefit ratio is

$$\text{RCB} = \frac{\mathcal{L}_{\text{tail}} - \mathcal{L}_{\text{condition}}}{\mathcal{L}_{\text{tail}} - \mathcal{L}_{\text{full}}}. \quad (4)$$

An RCB near one means that the condition recovers the measured loss benefit of dense context relative to the tail-only baseline. It is not a general correctness score and may slightly exceed one under finite-sample noise. We separately report annotated target coverage, active K/V fraction, physical K/V tokens, and operation length. This separation prevents a successful transport path from being mistaken for a successful learned router.

5 Does Selected Native K/V Carry the Needed Detail?

The first test removes routing ambiguity by selecting the annotated source. On synthetic data, dense context reaches loss 0.0105 and 100% accuracy, while the tail-only condition has loss 1.6999 and 50.6% accuracy. Oracle native-KV selection remains at 100% accuracy from 3 through 32 splits; at 32 splits its loss is 0.1194, whereas shuffled references give loss 2.7038. Selected cached state therefore carries information that the direct tail does not.

Natural-text-derived probes show the same transport property. Table 2 reports the fixed-source split-32 condition, before the later scale repair. Oracle selected K/V recovers nearly all dense-context loss benefit while exposing under 10% of accessible K/V. Shuffling the selected source destroys that benefit. These are controlled answer-code experiments, not end-to-end QA scores.

Table 2: Native-KV transport at split 32. Active fraction is measured over accessible reference K/V.

Probe	Full loss	Tail loss	Oracle loss	Oracle RCB	Active fraction
HotpotQA-derived	0.0027	1.7038	0.0060	0.996	0.092
QASPER-derived	0.0026	1.8545	0.0039	0.999	0.088

The initial split-64 result was much worse: oracle RCB fell to 0.546 on HotpotQA-derived and 0.748 on QASPER-derived examples. Because an oracle selector also failed, the router could not be the cause. The failure came from independently encoding very small chunks.

6 The Fragmentation Repair

The repair was architectural rather than a router hyperparameter change. Encoding eight consecutive reference units together raised split-64 oracle RCB from 0.546 to 0.996 on HotpotQA-derived text and from 0.748 to 0.999 on QASPER-derived text. Preserving global source positions then raised routed RCB from 0.706 to 0.865 and from 0.799 to 0.993, respectively. The final native-slicing path, which encodes contextual blocks and exposes smaller routing views over their K/V, reaches 0.876 and 0.994 in that historical split-64 diagnostic at roughly 10% active K/V.

Table 3: **Contextual native-K/V slicing repairs the split-64 transport failure.** The selector column separates payload fidelity from learned routing quality.

Encoding and coordinate treatment	Selector	HotpotQA-derived RCB	QASPER-derived RCB
Independent routing units	Oracle	0.546	0.748
Eight units encoded together	Oracle	0.996	0.999
Contextual blocks with global positions	Routed	0.865	0.993
Final contextual native-KV slicing	Routed	0.876	0.994

This result establishes a design rule: *encode at the granularity required for contextual state; route at the granularity required for sparse selection*. Chunk count by itself is not meaningful unless the encoding procedure is also specified.

7 Scaling: Breadth, Fraction, and Coverage

After the fragmentation repair, we scale each natural-text-derived source to 64, 128, and 256 addressable units. Two comparisons answer different questions. Fixed $k = 8$ asks whether the physical working set remains bounded as logical memory grows. Selection near a fixed 12.5% asks whether routing quality remains stable under comparable sparsity pressure.

Table 4: Small-tier scaling under two controls. Fixed- k coverage and RCB use $k = 8$; fixed-fraction coverage uses the measured k nearest 12.5% (8, 16, 32).

Probe	Units	Active fraction	Fixed- k coverage	Routed RCB	Fixed-fraction coverage
HotpotQA-derived	64	0.179	0.756	1.000	0.756
	128	0.089	0.713	1.000	0.772
	256	0.044	0.700	1.000	0.762
QASPER-derived	64	0.187	0.794	1.000	0.794
	128	0.097	0.800	1.000	0.803
	256	0.049	0.759	1.000	0.800

7.1 Fixed breadth: a nearly bounded physical working set

At fixed $k = 8$, candidate count grows from 63 to 255 while the small model materializes only 11–13 physical K/V tokens per example. The active fraction falls from .179 to .044 on the HotpotQA-derived probe and from .187 to .049 on the QASPER-derived probe. At 256 units this avoids 96.9–97.4% of reference K/V and 94.9–95.6% of total accessible K/V. Routed RCB remains 1.000 to the reported precision. This is the PRA systems law measured here: accessible memory grows while the selected native-K/V working set remains approximately flat.

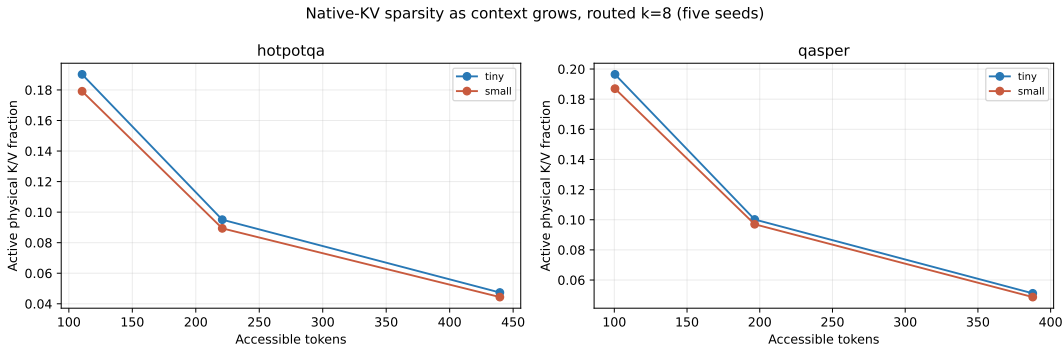


Figure 3: **Fixed $k = 8$ keeps physical selected K/V nearly flat as candidate memory grows.** The resulting active fraction falls to 4.4–4.9% at 256 addressable units.

7.2 Fixed fraction: routing quality under equal sparsity pressure

Fixed k becomes progressively harsher: the selected candidate fraction changes from 12.7% at 64 units to 6.3% at 128 and 3.1% at 256. Selecting the measured k nearest 12.5% removes that confound. Coverage remains 0.756/0.772/0.762 for HotpotQA-derived text and 0.794/0.803/0.800 for QASPER-derived text. Figure 4 shows every measured point; no values are interpolated.

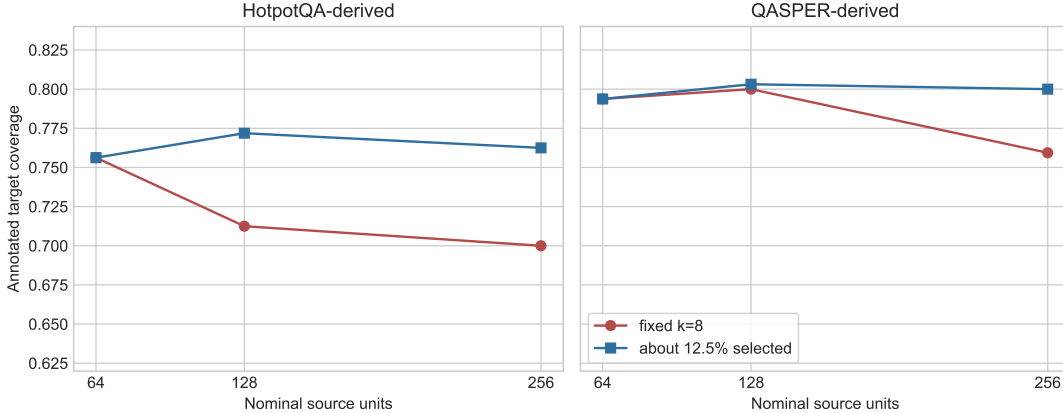


Figure 4: **Fixed- k decline partly reflects increasing sparsity, not intrinsic routing collapse.** Holding selection near 12.5% yields stable annotated target coverage through the measured 256-unit scale.

RCB and coverage answer different questions. RCB asks whether the selected K/V is sufficient for this controlled prediction. Coverage asks whether the annotated source was selected. A model can recover the answer when redundant or correlated evidence is present even if the annotated URI is absent. Accordingly, the perfect measured RCB at $k = 8$ should not be read as perfect routing.

7.3 Measured model-capacity effect

The same QASPER-derived $k = 8$ run separates transport from tested model capacity. Tiny-tier routed RCB declines from 1.000 at 64 units to 0.908 at 256, while the small tier remains 1.000. Its target coverage at 256 is also 0.388 versus 0.759. Increased capacity materially improves this measured high-scale operating point, but two small tiers do not support extrapolation to large models or prove that capacity alone solves routing.

8 Model-Bounded Logical Context

The scaling experiments above show sparse access, but they do not by themselves prove that every underlying operation respects a model limit. We therefore impose a hard $L_{\max} = 32$ token limit on every source-encoding and decode operation. The direct tail uses 8 tokens, the memory budget is 20, and 4 tokens are reserved for accounting safety. The logical prompt contains 192 tokens, of which 184 form an implicit addressable head. Encoding uses 16-token cores with up to four tokens of left context; the largest observed native operation is 20 tokens, or 62.5% of the declared limit.

Logical request: 192 tokens
184-token displaced <code>#_head</code> + 8-token direct tail
↓ head encoded in bounded blocks (largest source call: 20 tokens)
↓ compact routing selects full-detail K/V under a 20-token memory budget
↓ native attention constraint: 8 direct + L_M selected + 4 reserve ≤ 32

Figure 5: **A 192-token logical request executes without a native call longer than 20 tokens under the configured 32-token limit.** Logical history, displaced head, direct tail, materialized memory, and native operation length are distinct quantities.

Table 5: **Routed PRA improves on truncation while all measured native calls remain bounded.** Five seeds cover 80 held-out cases. Dense full context is a diagnostic outside the imposed deployment limit and the model’s shorter training distribution.

Condition	Loss	Accuracy	Target recall
Dense full diagnostic	1.7145	0.5125	–
Tail truncation	1.2287	0.6500	–
Independent blocks	1.7089	0.6375	0.2625
Shuffled/wrong memory	1.1205	0.6375	–
Routed PRA	1.0567	0.6875	0.525
Approximate oracle	0.8738	0.7125	1.000

Routed PRA improves loss by 0.1720 and accuracy by 3.75 percentage points relative to truncation, while all native calls remain within the 32-token limit. The approximate oracle shows remaining headroom. However, shuffled memory also improves loss over truncation, so this experiment demonstrates useful sparse access and bounded execution, not complete content-causal retrieval.

Increasing the memory budget from 4 to 20 tokens raises target recall from 0.325 to 0.525 and reduces loss from 1.2416 to 1.0567. Figure 6 makes the tradeoff explicit: the available materialization budget controls how much selected full-detail K/V can enter the bounded operation.

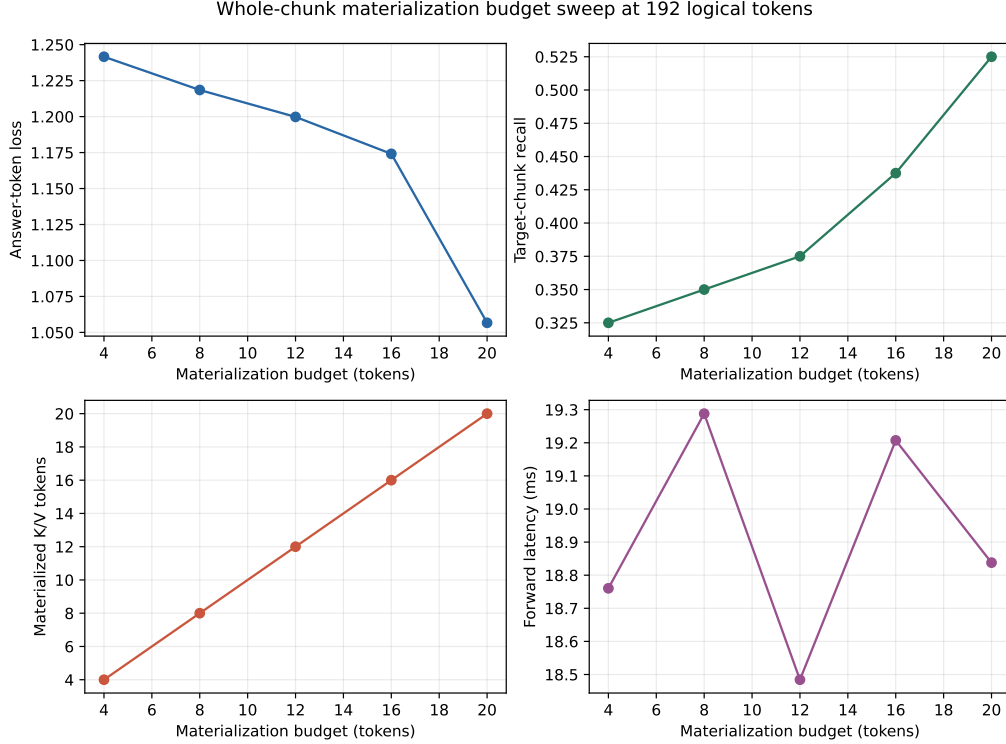


Figure 6: **The materialization budget exposes a quality–working-set frontier.** Assigning more of the 32-token operation budget to selected native K/V improves loss and target recall.

The same mechanism supports rollover during generation. In a 48-token generation smoke test, 11 rollovers migrate 44 historical tokens into addressable memory while the largest operation remains 20 tokens. This verifies the state transition; it is not a long-form generation quality result.

9 Implementation and Systems Measurements

The initial exact router repeatedly rebuilt Python objects and scored candidates in loops. The packed implementation normalizes candidate gists once, records offsets and ownership in tensors, and applies batched matrix products plus `torch.topk`. At 256 candidates, warm exact search takes 5.49–6.28 ms in the tiny tier and 11.46–11.56 ms in the small tier, compared with 563–1183 ms for the scalar path. The selected chunks are identical. This 94–103× microbenchmark improvement demonstrates that exact routing need not be Python-loop bound; it is not an end-to-end latency claim.

Persistent source K/V may also reside on CPU and move to the accelerator only after selection. At 256 units, this removes 0.734–3.420 MiB of measured source K/V from GPU residency and transfers only 24.6–87.9 KiB per example. The selected identities, materialized tokens, and loss match GPU-resident execution exactly. Selective transfer itself takes 4.0–8.9 ms; the complete warm-request path is 4.0–13.5 ms slower. Index memory remains on GPU at 260 KiB in the tiny tier and 1.03 MiB in the small tier. Figure 7 also shows why source-K/V savings should not be confused with whole-model peak allocation.

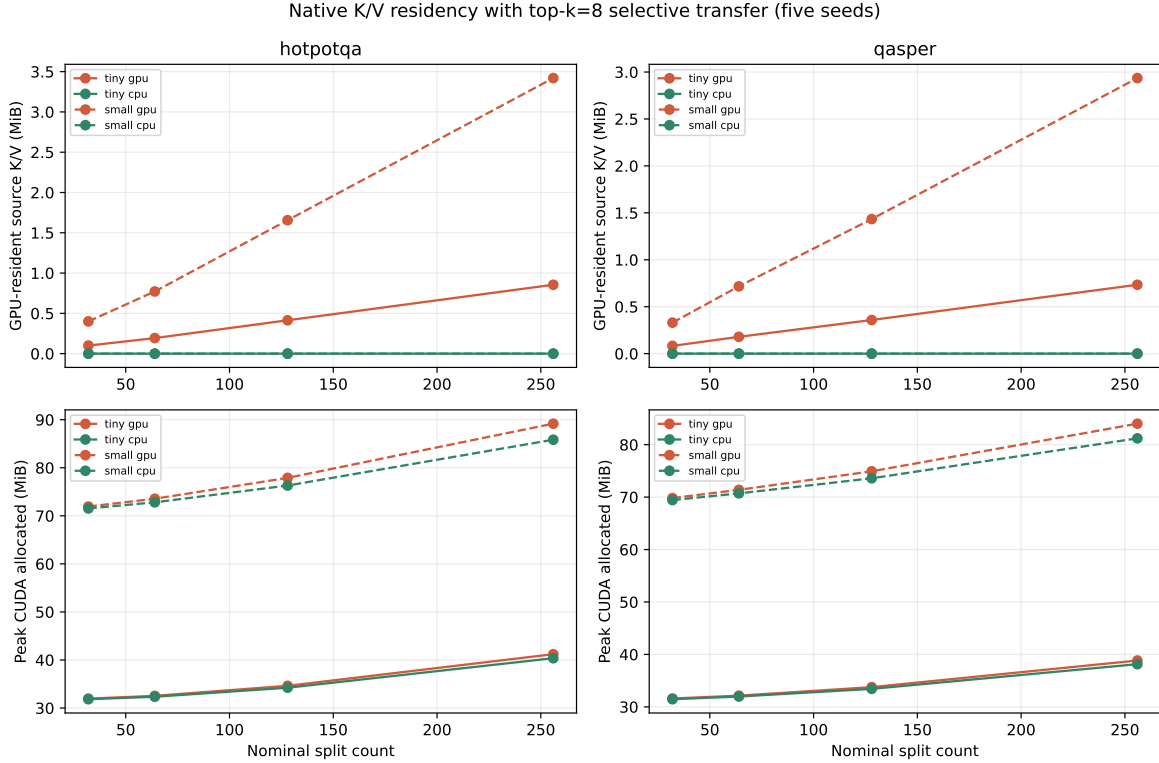


Figure 7: **CPU residency removes the full inactive source cache from GPU while preserving the sparse selected payload.** At 256 units the measured saving is .734–3.420 MiB; total peak CUDA allocation falls less because model parameters and other tensors remain resident.

Table 6: Systems measurements distinguish sparse-compute mechanics from unmeasured production serving claims.

Measurement	Established	Not established
Exact packed routing	256-way warm selection in 5.5–11.6 ms with scalar parity	Production decode latency or approximate-index scaling
CPU-resident K/V	Lower GPU residency with identical selected state and loss	Free storage or transfer-free attention
Bounded rollover	Continued generation without exceeding the declared operation limit	Long-form generation quality

10 Related Work by Changed Resource

Retrieved input. RAG, REALM, and RETRO retrieve external text or neighbors that a model then consumes [5, 3, 2]. PRA can use those retrieval results, but it operates after encoding: its addressable object is layer-native state and its immediate budget is the K/V active in attention.

Attention connectivity. Longformer and BigBird prescribe sparse connectivity over an admitted sequence. Reformer and Routing Transformer reduce attention cost through locality or content routing [1, 10, 4, 8]. PRA instead allows logical memory to remain outside the current model

operation and materializes selected token K/V only when needed.

Retained detail. Decode-cache methods such as H₂O, SnapKV, and ClusterKV change which token details remain available [11, 6, 7]. They are complementary when their retention policies preserve state PRA may later select. Memorizing Transformers and related external-memory models also expose prior state to a decoder [9]. PRA’s specific separation is among compact routing metadata, addressable full-detail native K/V, and a hard materialization budget.

11 Limitations, Safety, and Scope

The evidence is controlled. Models are small, targets are synthetic or answer-coded, and the natural-text probes are not unrestricted HotpotQA or QASPER evaluation. Five paired seeds establish directional consistency but offer limited statistical resolution. At five pairs, the smallest attainable exact two-sided sign-flip result is $p = 0.0625$.

Routing quality remains the main unresolved issue. Stable fixed-fraction target coverage through 256 units is encouraging, but coverage near 0.76–0.80 leaves substantial room for better selectors. The bounded-context router reaches only 0.525 recall, and the shuffled control shows that some gain is not source-specific. PRA makes sparse access possible; it does not make relevance estimation automatic.

Sparse selection alone reduces active attention, not necessarily GPU residency. If every inactive detail tensor remains on the accelerator, the external cache still grows with logical memory. The CPU-residency experiment measures one offload tradeoff, but it does not establish production transfer efficiency, asynchronous overlap, or arbitrary memory scale. Logical-memory support is likewise not dense full-context equivalence: unselected K/V is absent from the operation.

Position semantics are another independent constraint. Preserving source coordinates is necessary for the standalone absolute-position experiments, but this paper does not claim that a pre-trained positional geometry extrapolates to arbitrary logical offsets. The companion position paper studies RoPE-aware integration. Pretrained Hugging Face adapters, iterative multi-hop retrieval, multiple gists per chunk, and production serving are outside this paper’s evidence.

External K/V is sensitive derived state. Implementations must bind cache entries to request and authorization scope, prevent URI collisions across batch rows, define invalidation and retention, and avoid exposing routing traces across tenants. Implicit prompt-head memory is request-local even when every request uses the same internal name.

12 Conclusion

PRA separates addressable logical memory from the native K/V active in one transformer operation. Contextual encoding can remain coarse enough to preserve historical state while routing views remain fine enough for sparse selection. The gist is routing metadata; selected full-detail K/V is the attention payload.

The controlled evidence establishes both scaling laws through 256 units. At fixed k , physical selected K/V remains nearly flat and the active fraction falls to 4.4–4.9%. At a fixed selection fraction, target coverage remains stable near 0.76–0.80. A 192-token logical prompt also executes under a hard 32-token native limit, and CPU residency converts sparse selection into measured GPU-cache savings without changing loss or selected state.

This controlled scope leaves routing, pretrained positional integration, and optimized serving to future work. PRA nevertheless separates growing logical memory from per-operation active K/V.

A Secondary Ablations

Selection breadth. Increasing k improves coverage and raises active K/V. The correct operating point is therefore a frontier, not a universal constant. Fixed- k comparisons should always report the realized fraction, and fixed-fraction comparisons should disclose the measured k chosen at each candidate count.

Chunk overlap. Overlap can protect facts that cross routing boundaries, but it duplicates physical K/V and makes selected-token accounting differ from unique-token accounting. The main scaling tables report physical active tokens because those are materialized by the prototype.

Wrong-memory controls. Disabled, shuffled, irrelevant, and oracle conditions serve different purposes. Disabled memory measures the direct baseline; shuffled memory tests identity dependence while preserving payload volume; irrelevant memory tests distractor sensitivity; and oracle selection tests transport independently of routing. None alone is sufficient.

B Historical Cross-Attention Adaptation

An earlier V1 branch used a separately projected cross-attention-capable memory path. It is retained here for provenance, not as the architecture evaluated in the main paper. Across five paired seeds, freezing the pretrained backbone and adapting only that memory path reduced reference-conditioned loss from 6.3557 to 4.6720 in the full-PRA variant and from 6.3642 to 4.7296 in the hybrid variant. It preserved plain-language losses exactly at 4.5034 and 4.5500; direct joint tuning instead increased them by 0.1775 and 0.1562.

The result supports staged adaptation within that historical branch, but it does not establish a general pretrained-model result. Reference selection remained near chance (MRR 0.4202), and shuffled/oracle deltas were very small. The native-KV experiments in Sections 5–8 are the evidence for the present mechanism.

C Minimal Reimplementation Guide

The following pseudocode captures the default single-gist, native-KV path evaluated in this paper. The source map refers to repository commit 5b71e2e; line numbers are pinned to that snapshot so later edits do not silently move the referenced implementation.

```
encode_source(source, max_native_length):
    for contextual_block in bounded_contiguous_blocks(source):
        hidden = transformer(contextual_block)
        for layer in pra_layers:
            K, V = layer.project_kv(hidden[layer])
            commit_only_nonoverlapping_core(K, V, source_positions)
    for routing_chunk in views_over_committed_kv():
        gist = normalized_mean(routing_chunk.keys)
        index.add(gist, kv_offsets, owner)

attend(query, direct_kv, memory_budget):
    scores = packed_normalized_gists @ normalize(query)
```

```

ranked = exact_topk(scores)
selected = admit_whole_chunks(ranked, memory_budget)
memory_kv = gather_original_native_kv(selected)
assert len(direct_kv) + len(memory_kv) + reserve <= native_limit
return attention(query, concat(memory_kv, direct_kv))

```

Table 7: Code map for an independent implementation. Line ranges refer to commit [5b71e2e](#).

Concern	Source lines	Role
Native K/V projection	<code>attention.py:89--130</code>	Reuse a layer’s existing key and value projections without a separate memory projection.
Budget and materialization	<code>attention.py:155--329</code>	Admit whole selected chunks, gather their token K/V, and account for overlap and timing.
Direct + memory attention	<code>attention.py:331--430</code>	Concatenate selected memory with direct K/V under one score normalization and output projection.
Packed exact index	<code>memory.py:457--639</code>	Normalize and pack gists, preserve ownership and offsets, and score candidates tensorially.
Selection semantics	<code>memory.py:947--1188</code>	Apply breadth, hierarchy, and stable ranking rules and serialize traces.
Bounded source encoding	<code>model.py:430--630</code>	Encode contiguous blocks with left context and commit only each block’s core state.
Cache construction	<code>model.py:631--900</code>	Partition registered sources, generate gists, and associate routing views with layer-native K/V.
Row-private batch cache	<code>memory.py:1217--1306</code>	Keep reference ownership and implicit heads isolated between batch rows.

An implementation should pass four tests before quality evaluation: memory-disabled PRA must match its SelfAttention source model; native-all memory must match dense-context tail logits where the position regime is valid; oracle selection must transport the target state; and packed routing must return the same ordering as a scalar exhaustive implementation. It should then report candidate count, realized selection fraction, physical and unique K/V tokens, target coverage, operation length, index-build and warm search time, transfer bytes, and row ownership.

References

- [1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.
- [3] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.
- [4] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela.

- Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [6] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: <https://arxiv.org/abs/2404.14469>.
 - [7] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.
 - [8] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: <https://arxiv.org/abs/2003.05997>.
 - [9] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.
 - [10] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.
 - [11] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: <https://openreview.net/forum?id=RkRrPp7GK0>.